Formulario de cotización Jags en Java: pseudocódigo y explicación línea por línea

Para Lógica de Programación. Proyecto: `FormularioCotizacionJava`, paquete `com.mary.cotizacion`. Todo está en **un solo archivo, `Main.java`**, como tus ejercicios. Arriba va la ventana (la pantalla) y abajo, después del comentario `VALIDACIONES (la lógica)`, las funciones que revisan cada dato.

1. Pseudocódigo general

```
Algoritmo SolicitarCotizacion
    Definir NUMERO_WHATSAPP Como Cadena
    NUMERO_WHATSAPP <- "573217823068"

    // 1. ENTRADA
    Leer nombre, telefono, tipoCliente, servicio, ciudad, mensaje

    // 2. PROCESO
    formularioValido <- Verdadero
    Si validarNombre(nombre) <> "" Entonces
        Mostrar error de nombre
        formularioValido <- Falso
    FinSi
    Si validarTelefono(telefono) <> "" Entonces
        Mostrar error de teléfono
        formularioValido <- Falso
    FinSi
    Si validarTipo(tipoCliente) <> "" Entonces
        Mostrar error de tipo
        formularioValido <- Falso
    FinSi
    Si validarServicio(servicio) <> "" Entonces
        Mostrar error de servicio
        formularioValido <- Falso
    FinSi
    Si validarCiudad(ciudad) <> "" Entonces
        Mostrar error de ciudad
        formularioValido <- Falso
    FinSi
    Si validarMensaje(mensaje) <> "" Entonces
        Mostrar error de mensaje
        formularioValido <- Falso
    FinSi

    // 3. DECISIÓN
    Si formularioValido = Falso Entonces
        Escribir "Revisa los campos marcados en rojo."
    SiNo
        // 4. SALIDA
        texto <- armarMensaje(nombre, telefono, tipoCliente, servicio, ciudad, mensaje)
        url <- "https://wa.me/" + NUMERO_WHATSAPP + "?text=" + texto
        Abrir url en el navegador
        Escribir "Listo. Se abrió WhatsApp con tu cotización."
    FinSi
FinAlgoritmo
```

2. Pseudocódigo de las funciones

dejarSoloNumeros

```
Funcion numeros <- dejarSoloNumeros(texto)
    numeros <- ""
    i <- 0
    Mientras i < Longitud(texto) Hacer
        caracter <- texto[i]
        Si caracter >= "0" Y caracter <= "9" Entonces
            numeros <- numeros + caracter
        FinSi
        i <- i + 1
    FinMientras
FinFuncion
```

validarNombre

```
Funcion error <- validarNombre(nombre)
    Si nombre = "" Entonces
        error <- "Escribe tu nombre."
    SiNo Si Longitud(nombre) < 2 Entonces
        error <- "Escribe tu nombre con al menos 2 letras."
    SiNo Si tieneNumero(nombre) Entonces
        error <- "Escribe tu nombre con letras, sin números."
    SiNo
        error <- ""
    FinSi
FinFuncion
```

validarTelefono

```
Funcion error <- validarTelefono(escrito)
    Si escrito = "" Entonces
        error <- "Escribe tu número de celular."
    SiNo
        digitos <- dejarSoloNumeros(escrito)
        Si Longitud(digitos) = 12 Y digitos empieza con "57" Entonces
            digitos <- digitos sin los 2 primeros
        FinSi
        Si Longitud(digitos) = 10 Y digitos empieza con "3" Entonces
            error <- ""
        SiNo
            error <- "Debe ser un celular de 10 dígitos."
        FinSi
    FinSi
FinFuncion
```

validarTipo, validarServicio, validarCiudad, validarMensaje

```
Funcion error <- validarTipo(empresa, residencial)
    Si empresa O residencial Entonces error <- "" SiNo error <- "Indica empresa u hogar."
FinSi

Funcion error <- validarServicio(posicion)
```

```
    Si posicion = 0 Entonces error <- "Elige el servicio." SiNo error <- "" FinSi

Funcion error <- validarCiudad(ciudad)
    Si ciudad = "" Entonces error <- "Escribe la ciudad."
    SiNo Si Longitud(ciudad) < 2 O tieneNumero(ciudad) Entonces error <- "Escribe la ciudad
con letras."
    SiNo error <- "" FinSi

Funcion error <- validarMensaje(mensaje)
    Si Longitud(mensaje) > 300 Entonces error <- "Máximo 300 caracteres." SiNo error <- ""
FinSi
```

3. Main.java, parte de abajo: las validaciones línea por línea

dejarSoloNumeros

Línea de código	Qué hace
`public String dejarSoloNumeros(String texto) {`	Declara la función: recibe un texto y devuelve otro texto (`String`).
`String numeros = "";`	Crea un acumulador vacío donde se irán guardando los dígitos.
`int i = 0;`	Contador inicial, como en tu `RegistroEstudiantesWhile`.
`while (i < texto.length()) {`	Ciclo `while`: se repite mientras el contador no llegue al final del texto.
`char caracter = texto.charAt(i);`	Toma el carácter que está en la posición `i`.
`if (caracter >= '0' && caracter <= '9') {`	Pregunta si ese carácter es un dígito (entre 0 y 9).
`numeros = numeros + caracter;`	Si es dígito, lo pega al final del acumulador.
`i++;`	Suma 1 al contador para pasar a la siguiente posición. Sin esto el ciclo nunca termina.
`return numeros;`	Devuelve solo los números. Ej.: "+57 321-782" → "57321782".

tieneNumero

Línea de código	Qué hace
`public boolean tieneNumero(String texto) {`	Función que responde verdadero o falso (`boolean`).
`if (dejarSoloNumeros(texto).equals("")) {`	Reutiliza la función anterior: si al quitar todo menos los números queda vacío…
`return false;`	…el texto no tenía números.
`} else { return true; }`	Si no (quedó algo), sí tenía números.

validarNombre

Línea de código	Qué hace
`public String validarNombre(String nombre) {`	Recibe el nombre y devuelve "" si está bien o el texto del error.
`if (nombre.equals("")) {`	Revisa si está vacío. En Java los textos se comparan con `.equals`, no con `==`.
`return "Escribe tu nombre.";`	Devuelve el error y sale de la función.
`if (nombre.length() < 2) {`	Revisa si tiene menos de 2 letras.
`if (tieneNumero(nombre)) {`	Revisa si trae números (ej. "Ana2").
`return "";`	Si pasó todas las preguntas, devuelve vacío: no hay error.

validarTelefono

Línea de código	Qué hace
`if (escrito.equals("")) {`	Si no escribieron nada, devuelve el error de vacío.
`String digitos = dejarSoloNumeros(escrito);`	Quita espacios, +, guiones: deja solo los números.
`if (digitos.length() == 12 && digitos.substring(0, 2).equals("57")) {`	Si tiene 12 dígitos **y** los dos primeros son 57 (indicativo de Colombia)…
`digitos = digitos.substring(2);`	…le quita esos dos primeros dígitos.
`if (digitos.length() == 10 && digitos.charAt(0) == '3') {`	Un celular colombiano tiene 10 dígitos y empieza por 3.
`return "";`	Si cumple, no hay error.
`return "Debe ser un celular de 10 dígitos, como 321 782 3068.";`	Si no cumple, devuelve el error.

formatearTelefono

Línea de código	Qué hace
`String digitos = dejarSoloNumeros(escrito);`	Deja solo los números.
`if (digitos.length() == 12) { digitos = digitos.substring(2); }`	Si traía el 57, se lo quita.
`return "+57 " + digitos.substring(0, 3) + " " + ...;`	Lo devuelve ordenado en grupos: "+57 321 782 3068".

validarTipo, validarServicio, validarCiudad, validarMensaje

Línea de código	Qué hace
`if (empresa || residencial) {`	`||` es "o": basta con que uno de los dos esté marcado.
`if (posicion == 0) {`	La posición 0 de la lista es "Selecciona un servicio", o sea que no eligió nada.

Línea de código	Qué hace
`if (ciudad.length() < 2 || tieneNumero(ciudad)) {`	Error si es muy corta **o** si trae números.
`if (mensaje.length() > 300) {`	El mensaje es opcional; solo falla si pasa de 300 caracteres.

armarMensaje

Línea de código	Qué hace
`String texto = "Hola, quiero solicitar una cotización.\n\n";`	Empieza el texto. `\n` es un salto de línea.
`texto = texto + "Nombre: " + nombre + "\n";`	Concatena (une con `+`) cada dato con su título y un salto de línea.
`if (!mensaje.equals("")) {`	`!` es "no": si el mensaje NO está vacío…
`texto = texto + "\nMensaje: " + mensaje;`	…lo agrega al final.
`return texto;`	Devuelve el mensaje completo para WhatsApp.

4. Main.java, parte de arriba: la ventana línea por línea

Encabezado del archivo (antes de la clase)

Orden en Java: 1) `package`, 2) `import`, 3) la clase, 4) dentro de la clase, el `main`. El encabezado no se ejecuta, solo prepara. Por eso el `main` no puede ir en la línea 1.

Línea de código	Qué hace
`package com.mary.cotizacion;`	Dice en qué carpeta vive el archivo (`src/com/mary/cotizacion`). Siempre es la primera línea de código.
`import java.awt.*;`	Trae colores, fuentes y la forma de acomodar los elementos.
`import java.awt.event.ActionEvent; / ActionListener;`	El evento del clic y el que escucha el clic del botón Enviar.
`import java.net.URI;`	Sirve para abrir el link de WhatsApp.
`import java.net.URLEncoder;`	Convierte espacios y tildes para que el link no se dañe.
`import java.nio.charset.StandardCharsets;`	Usa UTF-8 para que la ñ y las tildes salgan bien.
`import javax.swing.*;`	Trae la ventana, las cajas de texto, los botones y las listas.
`/* ... */`	Comentario: Java lo ignora. Resume los 4 pasos: entrada, proceso, decisión, salida.

La clase y el punto de inicio (main)

Línea de código	Qué hace
`public class Main extends JFrame {`	La clase se llama `Main`, igual que el archivo `Main.java` (en Java deben llamarse igual). `extends JFrame` quiere decir que es una ventana.
`public static void main(String[] args) {`	**Punto de inicio del programa.** Va justo debajo de la clase: es lo primero que corre Java.
`Main ventana = new Main();`	Crea el objeto ventana, igual que `Scanner teclado = new Scanner(System.in);`.
`ventana.setVisible(true);`	La muestra en pantalla.

Variables de la ventana

Línea de código	Qué hace
`static final String NUMERO_WHATSAPP = "573217823068";`	Constante (no cambia): el número de Jags sin + ni espacios.
`static final Color AZUL = new Color(0x19, 0x57, 0xA1);`	Constantes de color con la paleta de Jags.
`JTextField txtNombre = new JTextField();`	Caja de texto para el nombre (igual teléfono y ciudad).
`JRadioButton rbEmpresa = new JRadioButton("Empresa");`	Botón de opción para escoger Empresa o Residencial.
`String[] servicios = { "Selecciona un servicio", ... };`	Arreglo con las opciones de la lista desplegable.
`JComboBox<String> cbServicio = new JComboBox<>(servicios);`	Lista desplegable que muestra ese arreglo.
`JTextArea txtMensaje = new JTextArea(3, 20);`	Área de texto de varias líneas para el mensaje.
`JLabel errNombre = crearError();`	Etiqueta roja vacía debajo de cada campo, para mostrar el error.

Constructor: arma la pantalla

Línea de código	Qué hace
`setTitle("Jags Technology · Solicita tu cotización");`	Título de la ventana.
`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`	Al cerrar la ventana, termina el programa.
`setSize(460, 720);`	Tamaño en píxeles: ancho 460, alto 720.
`JPanel panel = new JPanel(); panel.setLayout(new BoxLayout(...Y_AXIS));`	Panel principal que acomoda todo de arriba hacia abajo.
`JPanel encabezado = new JPanel(...); ... titulo, sub`	Franja azul oscura de arriba con el título y el número.
`ButtonGroup grupoTipo = new ButtonGroup();`	Agrupa los dos radio: así solo se puede marcar uno.

Línea de código	Qué hace
`agregarCampo(panel, "Nombre *", txtNombre, errNombre);`	Pone en el panel: etiqueta + caja + error. Se repite para cada campo.
`JButton btnEnviar = new JButton("Solicitar cotización");`	Crea el botón azul.
`btnEnviar.addActionListener(new ActionListener() { ... enviarCotizacion(); });`	**El evento:** cuando se oprime el botón, se llama a `enviarCotizacion()`. Es el botón "=" de la calculadora.
`add(new JScrollPane(panel));`	Mete el panel en la ventana, con barra de desplazamiento.

enviarCotizacion(): el corazón del programa

Línea de código	Qué hace
`String nombre = txtNombre.getText().trim();`	**Entrada:** lee lo que escribieron. `trim()` quita espacios al inicio y al final.
`boolean esEmpresa = rbEmpresa.isSelected();`	Guarda verdadero si está marcada la opción Empresa.
`int posServicio = cbServicio.getSelectedIndex();`	Guarda la posición elegida en la lista (0 = no eligió).
`String mensaje = txtMensaje.getText().trim().replace("\n", " ");`	Lee el mensaje y cambia los Enter por espacios.
`boolean formularioValido = true;`	**Bandera:** empieza suponiendo que todo está bien.
`formularioValido = revisar(errNombre, validarNombre(nombre), formularioValido);`	**Proceso:** llama a la función `validarNombre` (que está abajo en el mismo archivo), muestra el error si hay, y actualiza la bandera. Se repite por cada campo.
`if (formularioValido == false) {`	**Decisión:** si algún campo falló…
`lblEstado.setText("Revisa los campos marcados en rojo.");`	…muestra el aviso general…
`return;`	…y corta: no sigue a WhatsApp.
`String tipo; if (esEmpresa) { tipo = "Empresa"; } else { tipo = "Residencial"; }`	**Salida:** convierte la opción marcada en texto.
`String texto = armarMensaje(...);`	Llama a la función de abajo que arma el mensaje con todos los datos.
`String url = "https://wa.me/" + NUMERO_WHATSAPP + "?text=" + URLEncoder.encode(...);`	Arma el enlace de WhatsApp. `URLEncoder` convierte tildes, espacios y saltos de línea para que quepan en un enlace.
`try { Desktop.getDesktop().browse(new URI(url)); ... }`	Intenta abrir el enlace en el navegador y muestra "Listo".

Línea de código	Qué hace
`catch (Exception ex) { JOptionPane.showMessageDialog(...); }`	Si no se pudo abrir, muestra el mensaje en una ventanita. Es el manejo de errores, como en la división por cero.

Funciones de apoyo

Línea de código	Qué hace
`boolean revisar(JLabel etiquetaError, String error, boolean valido) {`	Recibe dónde mostrar el error, el error y cómo va la bandera.
`if (error.equals("")) { etiquetaError.setText(" "); return valido; }`	Si no hay error, limpia la etiqueta y deja la bandera como estaba.
`else { etiquetaError.setText(error); return false; }`	Si hay error, lo muestra en rojo y devuelve `false`.
`JLabel crearError() { ... l.setForeground(ROJO); ... }`	Fábrica de etiquetas rojas para los errores.
`void agregarCampo(JPanel panel, String texto, JComponent campo, JLabel error) {`	Arma una fila: etiqueta en negrita, el campo y su error debajo. Evita repetir el mismo código 6 veces.

5. Si te preguntan en clase

- **¿Por qué un solo archivo?** Así se parece a los ejercicios de clase: una sola clase `Main` con su `main`. Las validaciones son funciones dentro de la misma clase, por eso se llaman directo, sin crear otro objeto.
- **¿Qué es la bandera?** Una variable boolean que empieza en verdadero y cualquier error la pone en falso.
- **¿Dónde está el ciclo?** En `dejarSoloNumeros`, que recorre el texto con `for`.
- **¿Por qué `.equals` y no `==`?** Porque en Java los textos (String) se comparan por su contenido con `.equals`.
- **¿Qué hace el `return` dentro del `if`?** Sale de la función; por eso WhatsApp no se abre si hay errores.